

▼ TRANSFORMER NEURAL NETWORK MODEL

```
# TRANSFORMER NEURAL NETWORK MODEL

# 1. Import all necessary libraries

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.preprocessing import StandardScaler

from tensorflow.keras.layers import *
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras import regularizers

# 2. Upload Required files ( train, val, test)

from google.colab import files
uploaded = files.upload()

files_list = os.listdir()

def find_file(keyword):
    matches = [f for f in files_list if keyword in f.lower()]
    return matches[0]

train_file = find_file("augmented")
val_file = find_file("validation")
test_file = find_file("test")

# Load datasets
train_df = pd.read_csv(train_file)
val_df = pd.read_csv(val_file)
test_df = pd.read_csv(test_file)

# Split data
X_train = train_df.iloc[:, :-1].values
y_train = train_df.iloc[:, -1].values

X_val = val_df.iloc[:, :-1].values
y_val = val_df.iloc[:, -1].values

X_test = test_df.iloc[:, :-1].values
y_test = test_df.iloc[:, -1].values

# SCALING

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

# OPTIONAL ORIGINAL DATA
original_file = None
for f in files_list:
    if "original" in f.lower():
        original_file = f

if original_file is not None:
    original_df = pd.read_csv(original_file)

    X_real = scaler.transform(original_df.iloc[:, :-1].values)
    y_real = original_df.iloc[:, -1].values

    X_train = np.vstack((X_train, X_real))
```

```

y_train = np.concatenate((y_train, y_real))

# ENCODING

y_train = to_categorical(y_train)
y_val   = to_categorical(y_val)
y_test  = to_categorical(y_test)

# reshape
num_features = X_train.shape[1]

X_train = X_train.reshape((X_train.shape[0], num_features, 1))
X_val   = X_val.reshape((X_val.shape[0], num_features, 1))
X_test  = X_test.reshape((X_test.shape[0], num_features, 1))

# BUILD TRANSFORMER

def transformer_block(x):

    residual = Dense(32)(x)

    x_norm = LayerNormalization()(residual)

    attn = MultiHeadAttention(
        num_heads=2,
        key_dim=16,
        dropout=0.3
    )(x_norm, x_norm)

    x = Add()([residual, attn])
    x = LayerNormalization()(x)

    ff = Dense(
        64,
        activation='relu',
        kernel_regularizer=regularizers.l2(1e-4)
    )(x)

    ff = Dropout(0.3)(ff)

    ff = Dense(32)(ff)

    x = Add()([x, ff])

    return LayerNormalization()(x)

# MODEL

inputs = Input(shape=(num_features, 1))

x = GaussianNoise(0.02)(inputs)

x = Dense(32, activation="relu")(x)

x = transformer_block(x)
x = transformer_block(x)
x = transformer_block(x)

x = GlobalAveragePooling1D()(x)

x = Dense(
    64,
    activation="relu",
    kernel_regularizer=regularizers.l2(1e-4)
)(x)
x = Dropout(0.4)(x)

x = Dense(
    32,
    activation="relu",
    kernel_regularizer=regularizers.l2(1e-4)
)(x)
x = Dropout(0.3)(x)

outputs = Dense(y_train.shape[1], activation='softmax')(x)

```

```
model = tf.keras.Model(inputs, outputs)

# OPTIMIZER

optimizer = tf.keras.optimizers.Adam(
    learning_rate=1.5e-4,
    clipnorm=1.0
)

model.compile(
    optimizer=optimizer,
    loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.15),
    metrics=['accuracy']
)

model.summary()

# CALLBACKS

callbacks = [
    EarlyStopping(
        monitor='val_accuracy',
        patience=8,
        restore_best_weights=True
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=2,
        min_lr=1e-6,
        verbose=1
    )
]

# TRAINING

history = model.fit(
    X_train, y_train,
    epochs=70,
    batch_size=64,
    validation_data=(X_val, y_val),
    callbacks=callbacks,
    verbose=1
)

# EVALUATION

y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

accuracy = accuracy_score(y_true, y_pred_classes)
precision = precision_score(y_true, y_pred_classes, average='macro')
recall = recall_score(y_true, y_pred_classes, average='macro')
f1 = f1_score(y_true, y_pred_classes, average='macro')

print("\nTRANSFORMER PERFORMANCE")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")

# CONFUSION MATRIX

cm = confusion_matrix(y_true, y_pred_classes)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.title("Transformer Confusion Matrix")
plt.show()

# TRAINING CURVES
```

```
plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='val')
plt.title("Accuracy")
plt.legend()
plt.show()
```

```
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='val')
plt.title("Loss")
plt.legend()
plt.show()
```

SAVE MODEL

```
model.save("final_transformer_model.keras")
print("Transformer model saved")
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving test_set_scaled.csv to test_set_scaled.csv

Saving validation_set_scaled.csv to validation_set_scaled.csv

Saving preprocessed_augmented_dataset_scaled.csv to preprocessed_augmented_dataset_scaled.csv

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 19, 1)	0	-
gaussian_noise (GaussianNoise)	(None, 19, 1)	0	input_layer[0][0]
dense (Dense)	(None, 19, 32)	64	gaussian_noise[0...
dense_1 (Dense)	(None, 19, 32)	1,056	dense[0][0]
layer_normalization (LayerNormalizatio...	(None, 19, 32)	64	dense_1[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 19, 32)	4,224	layer_normalizat... layer_normalizat...
add (Add)	(None, 19, 32)	0	dense_1[0][0], multi_head_atten...
layer_normalizatio... (LayerNormalizatio...	(None, 19, 32)	64	add[0][0]
dense_2 (Dense)	(None, 19, 64)	2,112	layer_normalizat...
dropout_1 (Dropout)	(None, 19, 64)	0	dense_2[0][0]
dense_3 (Dense)	(None, 19, 32)	2,080	dropout_1[0][0]
add_1 (Add)	(None, 19, 32)	0	layer_normalizat... dense_3[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 19, 32)	64	add_1[0][0]
dense_4 (Dense)	(None, 19, 32)	1,056	layer_normalizat...
layer_normalizatio... (LayerNormalizatio...	(None, 19, 32)	64	dense_4[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 19, 32)	4,224	layer_normalizat... layer_normalizat...
add_2 (Add)	(None, 19, 32)	0	dense_4[0][0], multi_head_atten...
layer_normalizatio... (LayerNormalizatio...	(None, 19, 32)	64	add_2[0][0]
dense_5 (Dense)	(None, 19, 64)	2,112	layer_normalizat...
dropout_3 (Dropout)	(None, 19, 64)	0	dense_5[0][0]
dense_6 (Dense)	(None, 19, 32)	2,080	dropout_3[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 19, 32)	64	add_2[0][0], dense_6[0][0]

Abation Study

```
# =====
# ABLATION STUDY: TRANSFORMER FOR BEARING FAULT DETECTION
# =====

# !pip -q install tensorflow

import os
```

```

import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras.layers import (
    Input, Dense, Dropout, LayerNormalization, MultiHeadAttention,
    Add, GaussianNoise, GlobalAveragePooling1D
)
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras import regularizers
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.models import Model

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

# =====
# LOAD DATA
# =====

files_list = os.listdir()

def find_file(keyword):
    matches = [f for f in files_list if keyword in f.lower()]
    return matches[0]

train_file = find_file("augmented")
val_file = find_file("validation")
test_file = find_file("test")

print("Train:", train_file)
print("Val :", val_file)
print("Test :", test_file)

train_df = pd.read_csv(train_file)
val_df = pd.read_csv(val_file)
test_df = pd.read_csv(test_file)

X_train_raw = train_df.iloc[:, :-1].values
y_train_raw = train_df.iloc[:, -1].values

X_val_raw = val_df.iloc[:, :-1].values
y_val_raw = val_df.iloc[:, -1].values

X_test_raw = test_df.iloc[:, :-1].values
y_test_raw = test_df.iloc[:, -1].values

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_raw)
X_val_scaled = scaler.transform(X_val_raw)
X_test_scaled = scaler.transform(X_test_raw)

# Optional original-data augmentation
original_file = None
for f in files_list:
    if "original" in f.lower():
        original_file = f

if original_file is not None:
    original_df = pd.read_csv(original_file)
    X_real = scaler.transform(original_df.iloc[:, :-1].values)
    y_real = original_df.iloc[:, -1].values
    X_train_scaled = np.vstack((X_train_scaled, X_real))
    y_train_raw = np.concatenate((y_train_raw, y_real))

y_train = to_categorical(y_train_raw)
y_val = to_categorical(y_val_raw)
y_test = to_categorical(y_test_raw)
num_classes = y_train.shape[1]

num_features = X_train_scaled.shape[1]
X_train = X_train_scaled.reshape((X_train_scaled.shape[0], num_features, 1))
X_val = X_val_scaled.reshape((X_val_scaled.shape[0], num_features, 1))
X_test = X_test_scaled.reshape((X_test_scaled.shape[0], num_features, 1))

```

```

# class weights precomputed so ablation variant 10 can opt in to them
classes = np.unique(y_train_raw)
weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train_raw)
class_weights = dict(zip(classes, weights))

print("Shape:", X_train.shape, "| Classes:", num_classes)

# =====
# CONFIGURABLE MODEL BUILDER
# =====

DEFAULT_CONFIG = {
    "name": "0_baseline_exact",
    "use_gaussian_noise": True,
    "gaussian_noise_std": 0.02,
    "num_transformer_blocks": 3,
    "num_heads": 2,
    "key_dim": 16,
    "attn_dropout": 0.3,
    "ff_dropout": 0.3,
    "use_dropout": True,          # master switch for attn/ff/head dropout
    "head_dropout": [0.4, 0.3],  # Dropout after Dense(64), Dropout after Dense(32)
    "use_l2": True,
    "l2_value": 1e-4,
    "use_dense_head": True,
    "dense_head_units": [64, 32],
    "learning_rate": 1.5e-4,
    "clipnorm": 1.0,
    "label_smoothing": 0.15,
    "use_lr_schedule": True,
    "lr_factor": 0.5,
    "lr_patience": 2,
    "early_stop_patience": 8,
    "use_class_weights": False,   # baseline
    "metric_average": "macro",
}

def reg(cfg):
    return regularizers.l2(cfg["l2_value"]) if cfg["use_l2"] else None

def transformer_block(x, cfg):
    residual = Dense(32)(x)
    x_norm = LayerNormalization()(residual)

    attn_dropout = cfg["attn_dropout"] if cfg["use_dropout"] else 0.0
    attn = MultiHeadAttention(
        num_heads=cfg["num_heads"],
        key_dim=cfg["key_dim"],
        dropout=attn_dropout,
    )(x_norm, x_norm)

    x = Add()([residual, attn])
    x = LayerNormalization()(x)

    ff = Dense(64, activation='relu', kernel_regularizer=reg(cfg))(x)
    if cfg["use_dropout"]:
        ff = Dropout(cfg["ff_dropout"])(ff)
    ff = Dense(32)(ff)

    x = Add()([x, ff])
    return LayerNormalization()(x)

def build_model(cfg):
    inputs = Input(shape=(num_features, 1))
    x = inputs

    if cfg["use_gaussian_noise"]:
        x = GaussianNoise(cfg["gaussian_noise_std"])(x)

    x = Dense(32, activation="relu")(x)

    for _ in range(cfg["num_transformer_blocks"]):
        x = transformer_block(x, cfg)

    x = GlobalAveragePooling1D()(x)

    if cfg["use_dense_head"]:
        for units, drop_rate in zip(cfg["dense_head_units"], cfg["head_dropout"]):
            x = Dense(units, activation="relu", kernel_regularizer=reg(cfg))(x)

```

```

        if cfg["use_dropout"]:
            x = Dropout(drop_rate)(x)

    outputs = Dense(num_classes, activation='softmax')(x)

    model = Model(inputs, outputs)

    opt_kwargs = {"learning_rate": cfg["learning_rate"]}
    if cfg["clipnorm"] is not None:
        opt_kwargs["clipnorm"] = cfg["clipnorm"]
    optimizer = Adam(**opt_kwargs)

    loss = tf.keras.losses.CategoricalCrossentropy(label_smoothing=cfg["label_smoothing"])
    model.compile(optimizer=optimizer, loss=loss, metrics=['accuracy'])
    return model

def get_callbacks(cfg):
    callbacks = [EarlyStopping(monitor='val_accuracy', patience=cfg["early_stop_patience"],
                               restore_best_weights=True)]
    if cfg["use_lr_schedule"]:
        callbacks.append(ReduceLROnPlateau(monitor='val_loss', factor=cfg["lr_factor"],
                                             patience=cfg["lr_patience"], min_lr=1e-6))
    return callbacks

def run_experiment(cfg, epochs=70, batch_size=64, verbose=0):
    tf.keras.backend.clear_session()
    tf.random.set_seed(SEED)

    model = build_model(cfg)
    cw = class_weights if cfg["use_class_weights"] else None

    history = model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_data=(X_val, y_val),
        callbacks=get_callbacks(cfg),
        class_weight=cw,
        verbose=verbose,
    )

    y_pred = model.predict(X_test, verbose=0)
    y_pred_classes = np.argmax(y_pred, axis=1)
    y_true = np.argmax(y_test, axis=1)
    avg = cfg["metric_average"]

    metrics = {
        "variant": cfg["name"],
        "accuracy": accuracy_score(y_true, y_pred_classes),
        "precision": precision_score(y_true, y_pred_classes, average=avg, zero_division=0),
        "recall": recall_score(y_true, y_pred_classes, average=avg, zero_division=0),
        "f1_score": f1_score(y_true, y_pred_classes, average=avg, zero_division=0),
        "epochs_run": len(history.history['accuracy']),
        "final_val_accuracy": history.history['val_accuracy'][-1],
        "final_val_loss": history.history['val_loss'][-1],
        "trainable_params": model.count_params(),
    }
    return metrics, history

# =====
# DEFINE ABLATION VARIANTS
# =====

def make_cfg(overrides):
    cfg = dict(DEFAULT_CONFIG)
    cfg.update(overrides)
    return cfg

ablation_variants = [
    make_cfg({"name": "0_baseline_exact"}),

    make_cfg({"name": "1_no_gaussian_noise", "use_gaussian_noise": False}),

    make_cfg({"name": "2_no_label_smoothing", "label_smoothing": 0.0}),

    make_cfg({"name": "3_no_l2_regularization", "use_l2": False}),

    make_cfg({"name": "4_single_transformer_block", "num_transformer_blocks": 1}),

```

```

make_cfg({"name": "5_no_dropout", "use_dropout": False}),

make_cfg({"name": "6_four_attention_heads", "num_heads": 4}),

make_cfg({"name": "7_no_gradient_clipping", "clipnorm": None}),

make_cfg({
    "name": "8_fixed_lr_no_schedule",
    "learning_rate": 5e-4,
    "use_lr_schedule": False,
}),

make_cfg({"name": "9_no_dense_head_after_gap", "use_dense_head": False}),

make_cfg({"name": "10_with_class_weights", "use_class_weights": True}),
]

# =====
# RUN ALL EXPERIMENTS
# =====
# NOTE: epochs=70 matches the original but 11 full runs at that length
# is slow (original took ~40 epochs w/ early stop, ~7-20s/epoch).
# EPOCHS below trims the sweep; restore to 70 for a final run.

EPOCHS = 40

results = []
histories = {}

for cfg in ablation_variants:
    print("\n" + "=" * 70)
    print(f"Running variant: {cfg['name']}")
    print("=" * 70)
    metrics, history = run_experiment(cfg, epochs=EPOCHS, batch_size=64, verbose=1)
    results.append(metrics)
    histories[cfg["name"]] = history.history
    print(f"    -> Accuracy: {metrics['accuracy']:.4f} | F1: {metrics['f1_score']:.4f} "
          f"| Epochs: {metrics['epochs_run']} | Params: {metrics['trainable_params']:,}")

results_df = pd.DataFrame(results).sort_values("f1_score", ascending=False).reset_index(drop=True)

print("\n" + "=" * 70)
print("ABLATION STUDY RESULTS (sorted by F1 score, macro-averaged)")
print("=" * 70)
print(results_df.to_string(index=False))

results_df.to_csv("transformer_ablation_results.csv", index=False)
with open("transformer_ablation_histories.json", "w") as f:
    json.dump(histories, f)

print("\nSaved: transformer_ablation_results.csv, transformer_ablation_histories.json")

# =====
# VISUALIZATIONS
# =====

baseline_row = results_df[results_df["variant"] == "0_baseline_exact"].iloc[0]

plt.figure(figsize=(14, 6))
metrics_to_plot = ["accuracy", "precision", "recall", "f1_score"]
x = np.arange(len(results_df))
width = 0.2
for i, metric in enumerate(metrics_to_plot):
    plt.bar(x + i * width, results_df[metric], width, label=metric)
plt.xticks(x + width * 1.5, results_df["variant"], rotation=45, ha='right')
plt.ylabel("Score")
plt.title("Transformer Ablation Study: Metric Comparison Across Variants", fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig("transformer_ablation_metric_comparison.png", dpi=150)
plt.show()

plt.figure(figsize=(12, 6))
delta_f1 = results_df["f1_score"] - baseline_row["f1_score"]
colors = ['#d62728' if v < 0 else '#2ca02c' for v in delta_f1]
plt.barh(results_df["variant"], delta_f1, color=colors)
plt.axvline(0, color='black', linewidth=0.8)
plt.xlabel("Change in F1 Score vs. Baseline")
plt.title("Transformer: Impact of Removing/Changing Each Component (relative to baseline)",
          fontsize=12, fontweight='bold')

```



```

        fontsize=13, fontweight='bold',
plt.grid(True, axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig("transformer_ablation_delta_from_baseline.png", dpi=150)
plt.show()

plt.figure(figsize=(14, 6))
for name, hist in histories.items():
    plt.plot(hist['val_accuracy'], label=name, alpha=0.8)
plt.title("Transformer Validation Accuracy Curves – All Variants", fontsize=14, fontweight='bold')
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend(fontsize=8, loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("transformer_ablation_val_accuracy_curves.png", dpi=150)
plt.show()

print("\nSaved plots: transformer_ablation_metric_comparison.png, "
      "transformer_ablation_delta_from_baseline.png, transformer_ablation_val_accuracy_curves.png")

# =====
# SUMMARY
# =====
print("\n" + "=" * 70)
print("SUMMARY: Component impact (F1 delta vs baseline, macro-averaged)")
print("=" * 70)
summary = results_df.copy()
summary["f1_delta_vs_baseline"] = summary["f1_score"] - baseline_row["f1_score"]
summary = summary[summary["variant"] != "0_baseline_exact"]
summary = summary.sort_values("f1_delta_vs_baseline")
print(summary[["variant", "f1_delta_vs_baseline", "trainable_params"]].to_string(index=False))

```

```

Train: preprocessed_augmented_dataset_scaled.csv
Val : validation_set_scaled.csv
Test : test_set_scaled.csv
Shape: (100000, 19, 1) | Classes: 10

```

```

=====
Running variant: 0_baseline_exact
=====

```

```

Epoch 1/40
1563/1563 ----- 45s 14ms/step - accuracy: 0.5446 - loss: 1.6014 - val_accuracy: 0.7688 - val_loss:
Epoch 2/40
1563/1563 ----- 10s 7ms/step - accuracy: 0.7365 - loss: 1.2843 - val_accuracy: 0.7719 - val_loss:
Epoch 3/40
1563/1563 ----- 20s 6ms/step - accuracy: 0.7837 - loss: 1.2119 - val_accuracy: 0.8062 - val_loss:
Epoch 4/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.8103 - loss: 1.1676 - val_accuracy: 0.8438 - val_loss:
Epoch 5/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.8293 - loss: 1.1358 - val_accuracy: 0.8625 - val_loss:
Epoch 6/40
1563/1563 ----- 11s 7ms/step - accuracy: 0.8456 - loss: 1.1090 - val_accuracy: 0.8875 - val_loss:
Epoch 7/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.8580 - loss: 1.0861 - val_accuracy: 0.8844 - val_loss:
Epoch 8/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.8715 - loss: 1.0641 - val_accuracy: 0.8969 - val_loss:
Epoch 9/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.8808 - loss: 1.0462 - val_accuracy: 0.9094 - val_loss:
Epoch 10/40
1563/1563 ----- 10s 7ms/step - accuracy: 0.8899 - loss: 1.0307 - val_accuracy: 0.9156 - val_loss:
Epoch 11/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.8954 - loss: 1.0165 - val_accuracy: 0.9250 - val_loss:
Epoch 12/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.9019 - loss: 1.0034 - val_accuracy: 0.9250 - val_loss:
Epoch 13/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.9051 - loss: 0.9926 - val_accuracy: 0.9094 - val_loss:
Epoch 14/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.9135 - loss: 0.9781 - val_accuracy: 0.9156 - val_loss:
Epoch 15/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.9141 - loss: 0.9733 - val_accuracy: 0.9125 - val_loss:
Epoch 16/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.9159 - loss: 0.9683 - val_accuracy: 0.9094 - val_loss:
Epoch 17/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.9180 - loss: 0.9633 - val_accuracy: 0.9219 - val_loss:
Epoch 18/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.9215 - loss: 0.9578 - val_accuracy: 0.9156 - val_loss:
Epoch 19/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.9248 - loss: 0.9521 - val_accuracy: 0.9094 - val_loss:
-> Accuracy: 0.8848 | F1: 0.8801 | Epochs: 19 | Params: 33,578

```

```

=====
Running variant: 1_no_gaussian_noise
=====

```

```

Epoch 1/40
1563/1563 ----- 44s 14ms/step - accuracy: 0.5207 - loss: 1.6308 - val_accuracy: 0.7250 - val_loss:
Epoch 2/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.7358 - loss: 1.2826 - val_accuracy: 0.7844 - val_loss:
Epoch 3/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.7884 - loss: 1.2003 - val_accuracy: 0.8188 - val_loss:
Epoch 4/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.8176 - loss: 1.1518 - val_accuracy: 0.8344 - val_loss:
Epoch 5/40
1563/1563 ----- 10s 6ms/step - accuracy: 0.8344 - loss: 1.1224 - val_accuracy: 0.8406 - val_loss:
Epoch 6/40

```

Final Transformer Model

```

# =====
# FINAL TRANSFORMER MODEL FOR BEARING FAULT DETECTION
# =====

# !pip -q install tensorflow

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras.layers import (
    Input, Dense, Dropout, LayerNormalization, MultiHeadAttention,
    Add, GlobalAveragePooling1D
)
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

```

```

from tensorflow.keras.optimizers import Adam
from tensorflow.keras.models import Model

from google.colab import files
uploaded = files.upload()

# =====
# LOAD DATA
# =====

files_list = os.listdir()

def find_file(keyword):
    matches = [f for f in files_list if keyword in f.lower()]
    return matches[0] if matches else None

train_file = find_file("augmented")
val_file = find_file("validation")
test_file = find_file("test")

if None in [train_file, val_file, test_file]:
    raise FileNotFoundError("Missing required CSV files.")

train_df = pd.read_csv(train_file)
val_df = pd.read_csv(val_file)
test_df = pd.read_csv(test_file)

X_train = train_df.iloc[:, :-1].values
y_train_raw = train_df.iloc[:, -1].values

X_val = val_df.iloc[:, :-1].values
y_val_raw = val_df.iloc[:, -1].values

X_test = test_df.iloc[:, :-1].values
y_test_raw = test_df.iloc[:, -1].values

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

# Optional original-data augmentation
original_file = None
for f in files_list:
    if "original" in f.lower():
        original_file = f

if original_file is not None:
    original_df = pd.read_csv(original_file)
    X_real = scaler.transform(original_df.iloc[:, :-1].values)
    y_real = original_df.iloc[:, -1].values
    X_train = np.vstack((X_train, X_real))
    y_train_raw = np.concatenate((y_train_raw, y_real))

# Class weights (baseline omitted these; ablation showed +1.54% F1)
classes = np.unique(y_train_raw)
weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train_raw)
class_weights = dict(zip(classes, weights))

y_train = to_categorical(y_train_raw)
y_val = to_categorical(y_val_raw)
y_test = to_categorical(y_test_raw)

num_features = X_train.shape[1]
X_train = X_train.reshape((X_train.shape[0], num_features, 1))
X_val = X_val.reshape((X_val.shape[0], num_features, 1))
X_test = X_test.reshape((X_test.shape[0], num_features, 1))

# =====
# FINAL TRANSFORMER MODEL
# =====

def transformer_block(x):
    residual = Dense(32)(x)
    x_norm = LayerNormalization()(residual)

    attn = MultiHeadAttention(
        num_heads=4,
        key_dim=16,
        dropout=0.3,
    )(x_norm, x_norm)

```

```

    x = Add()([residual, attn])
    x = LayerNormalization()(x)

    ff = Dense(64, activation='relu')(x)
    ff = Dropout(0.3)(ff)
    ff = Dense(32)(ff)

    x = Add()([x, ff])
    return LayerNormalization()(x)

inputs = Input(shape=(num_features, 1))

x = Dense(32, activation="relu")(inputs)

x = transformer_block(x)
x = transformer_block(x)
x = transformer_block(x)

x = GlobalAveragePooling1D()(x)

outputs = Dense(y_train.shape[1], activation='softmax')(x)

model = Model(inputs, outputs)

optimizer = Adam(learning_rate=5e-4)

model.compile(
    optimizer=optimizer,
    loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.15),
    metrics=['accuracy']
)

model.summary()

# =====
# CALLBACKS
# =====

callbacks = [
    EarlyStopping(
        monitor='val_accuracy',
        patience=8,
        restore_best_weights=True
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=2,
        min_lr=1e-6,
        verbose=1
    )
]

# =====
# TRAIN FINAL MODEL
# =====

history = model.fit(
    X_train, y_train,
    epochs=70,
    batch_size=64,
    validation_data=(X_val, y_val),
    callbacks=callbacks,
    class_weight=class_weights,
    verbose=1
)

# =====
# EVALUATION
# =====

y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

accuracy = accuracy_score(y_true, y_pred_classes)
precision = precision_score(y_true, y_pred_classes, average='macro')
recall = recall_score(y_true, y_pred_classes, average='macro')
f1 = f1_score(y_true, y_pred_classes, average='macro')

print("\nFINAL TRANSFORMER PERFORMANCE")
print(f"Accuracy : {accuracy:.4f}")

```

```

print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")

# =====
# CONFUSION MATRIX (PLOT + PRINT)
# =====

cm = confusion_matrix(y_true, y_pred_classes)

# Print numerical confusion matrix
print("\nConfusion Matrix (counts):")
print(cm)

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
             xticklabels=classes, yticklabels=classes)
plt.title("Final Transformer Confusion Matrix")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.savefig("final_transformer_confusion_matrix.png", dpi=150)
plt.show()

# Print detailed classification report
print("\nClassification Report:")
print(classification_report(y_true, y_pred_classes, target_names=[str(c) for c in classes]))

# =====
# TRAINING CURVES
# =====

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='val')
plt.title("Final Transformer Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='val')
plt.title("Final Transformer Loss")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("final_transformer_curves.png", dpi=150)
plt.show()

# =====
# SAVE FINAL MODEL
# =====

model.save("final_transformer_model.keras")
print("\nFinal model saved to: final_transformer_model.keras")

1563/1563 ————— 10s 6ms/step - accuracy: 0.9554 - loss: 0.8158 - val_accuracy: 0.9125 - val_loss:
Epoch 9/40
1563/1563 ————— 10s 6ms/step - accuracy: 0.9599 - loss: 0.8060 - val_accuracy: 0.9156 - val_loss:
Epoch 10/40
1563/1563 ————— 10s 6ms/step - accuracy: 0.9641 - loss: 0.7984 - val_accuracy: 0.9062 - val_loss:
Epoch 11/40
1563/1563 ————— 10s 6ms/step - accuracy: 0.9678 - loss: 0.7915 - val_accuracy: 0.9125 - val_loss:
Epoch 12/40
1563/1563 ————— 10s 6ms/step - accuracy: 0.9739 - loss: 0.7784 - val_accuracy: 0.9031 - val_loss:
Epoch 13/40
1563/1563 ————— 10s 6ms/step - accuracy: 0.9754 - loss: 0.7745 - val_accuracy: 0.9031 - val_loss:
Epoch 14/40
1563/1563 ————— 9s 6ms/step - accuracy: 0.9791 - loss: 0.7674 - val_accuracy: 0.9125 - val_loss: 0
Epoch 15/40
1563/1563 ————— 10s 6ms/step - accuracy: 0.9795 - loss: 0.7661 - val_accuracy: 0.9156 - val_loss:
-> Accuracy: 0.8970 | F1: 0.8941 | Epochs: 15 | Params: 33,578

=====
Running variant: 6_four_attention_heads
=====

Epoch 1/40
1563/1563 ————— 47s 15ms/step - accuracy: 0.5441 - loss: 1.5978 - val_accuracy: 0.7688 - val_loss:
Epoch 2/40
1563/1563 ————— 10s 6ms/step - accuracy: 0.7494 - loss: 1.2627 - val_accuracy: 0.7906 - val_loss:
Epoch 3/40
1563/1563 ————— 11s 7ms/step - accuracy: 0.8000 - loss: 1.1852 - val_accuracy: 0.8281 - val_loss:

```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

1563/1563 name. 11s 7ms/step - accuracy: 0.8285 - loss: 1.1415 - val_accuracy: 0.8375 - val_loss: 1.0570

1563/1563 test_set_scaled.csv to test_set_scaled (1).csv

1563/1563 validation_set_scaled.csv to validation_set_scaled (1).csv loss: 1.1079 - val_accuracy: 0.8500 - val_loss: 1.0570

1563/1563 preprocessed_augmented_dataset_scaled.csv to preprocessed_augmented_dataset_scaled (1).csv

1563/1563 functional_1 10s 7ms/step - accuracy: 0.8635 - loss: 1.0794 - val_accuracy: 0.8938 - val_loss: 1.0570

Epoch 7/40

name	Output	Time	Param#	Connected to	loss	val_accuracy	val_loss
Epoch 8/40					1.0570	0.8813	
1563/1563 layer_1	(None, 10)	10s 6ms/step	9	0	0.8891	0.9000	
Epoch 9/40							
1563/1563 layer_1	(None, 10)	11s 7ms/step	64	input_layer_1[0]...	0.8975	0.8875	
Epoch 10/40							
1563/1563 dense_12	(None, 10)	10s 7ms/step	1,056	dense_11[0][0]	0.9039	0.9000	
Epoch 11/40							
1563/1563 layer_normalization...	(None, 10)	11s 7ms/step	64	dense_12[0][0]	0.9107	0.8975	
Epoch 12/40							
1563/1563 layer_normalization...	(None, 10)	11s 7ms/step	64	dense_12[0][0]	0.9162	0.9062	
Epoch 13/40							
1563/1563 layer_normalization...	(None, 10)	11s 7ms/step	64	dense_12[0][0]	0.9162	0.9062	